

实验三报告

唐宇恩 251220100

2026 年 5 月 7 日

1 4 位二进制同步计数器

1.1 整体方案设计

本实验采用“独立逻辑门 + 4 位 D 触发器”的方式实现同步计数器。计数器具有同步置数、异步清零、使能计数和进位输出等功能：当 $CLR = 0$ 时，输出立即清零；当 $LD = 0$ 时，在时钟上升沿把输入端 $D_3 \sim D_0$ 同步装入；当 $ENP = ENT = 1$ 时，计数器按二进制加一方式递增，并在状态为 1111 且继续计数时输出进位信号 RCO 。

端口定义：

端口符号	端口功能
输入端 CLK	时钟输入端
输入端 CLR	异步清零端，低电平有效
输入端 LD	同步装载端，低电平有效
输入端 $D_0 \sim D_3$	预置数据输入端
输入端 ENP, ENT	计数使能端
输出端 $Q_0 \sim Q_3$	4 位计数结果输出端
输出端 RCO	进位输出端

电路中利用组合逻辑生成每一位触发器的下一状态，并将最低位进位逐级传递到高位，保证四位状态在同一时钟沿统一更新。

1.2 实验电路图

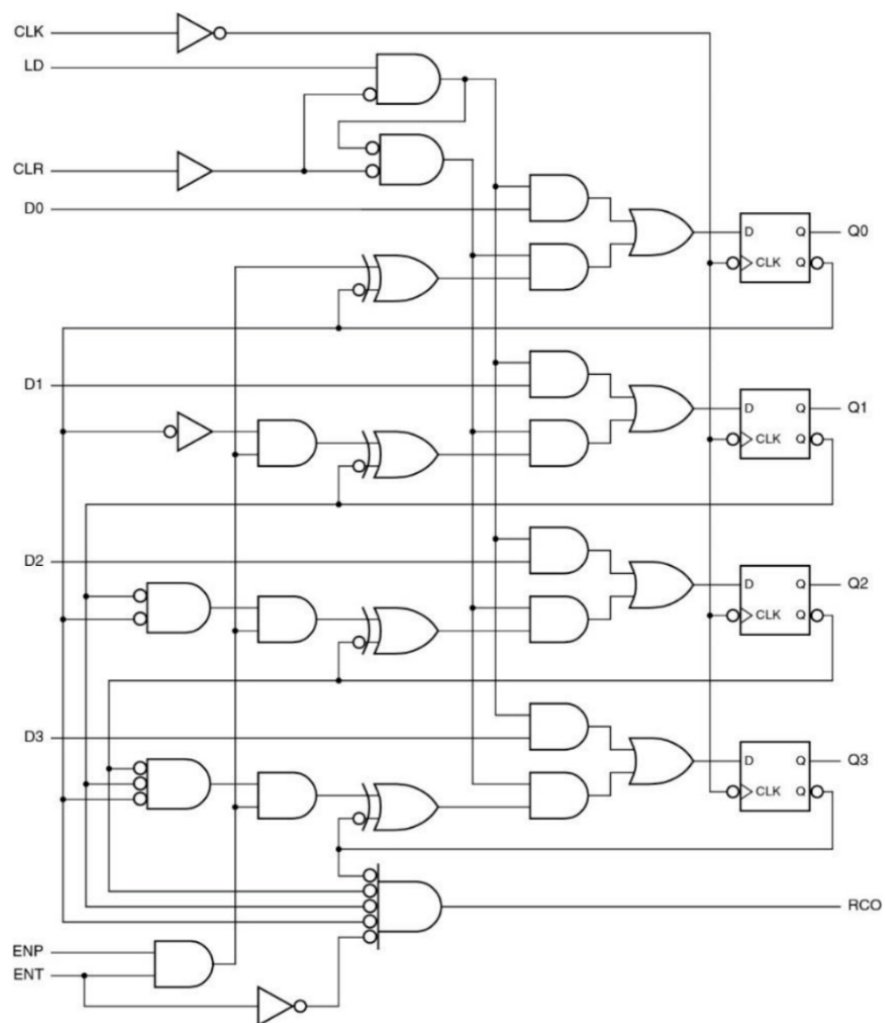


图 1: 4 位二进制同步计数器原理图

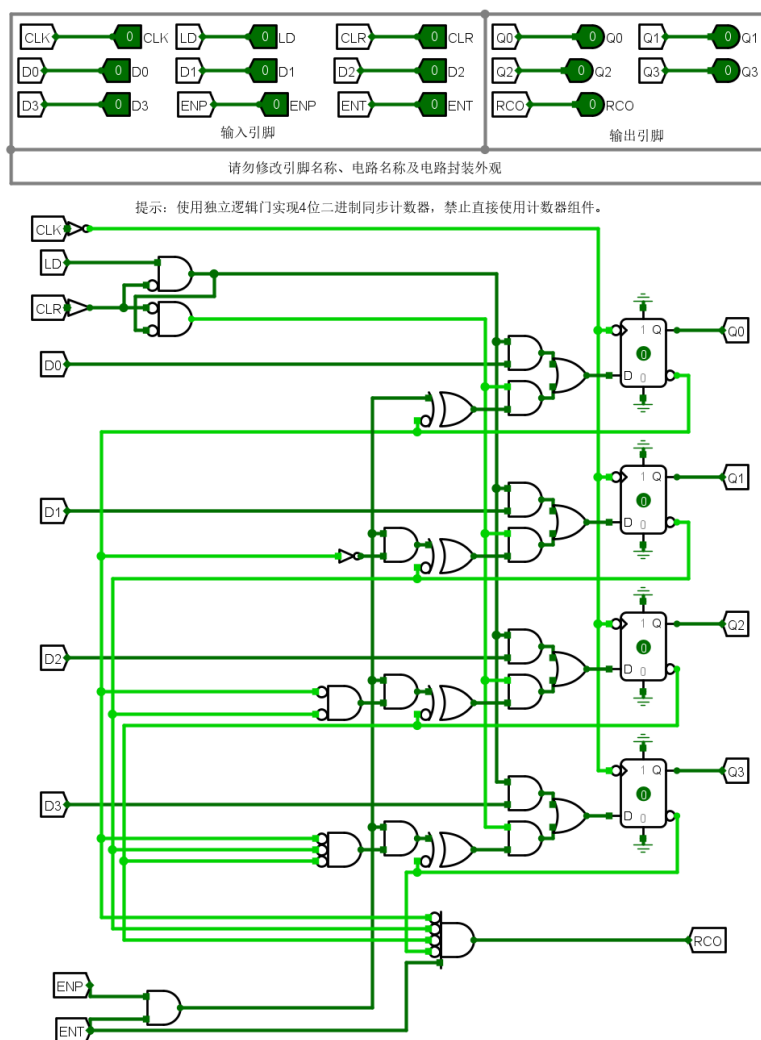


图 2: 4 位二进制同步计数器电路图

1.3 仿真测试

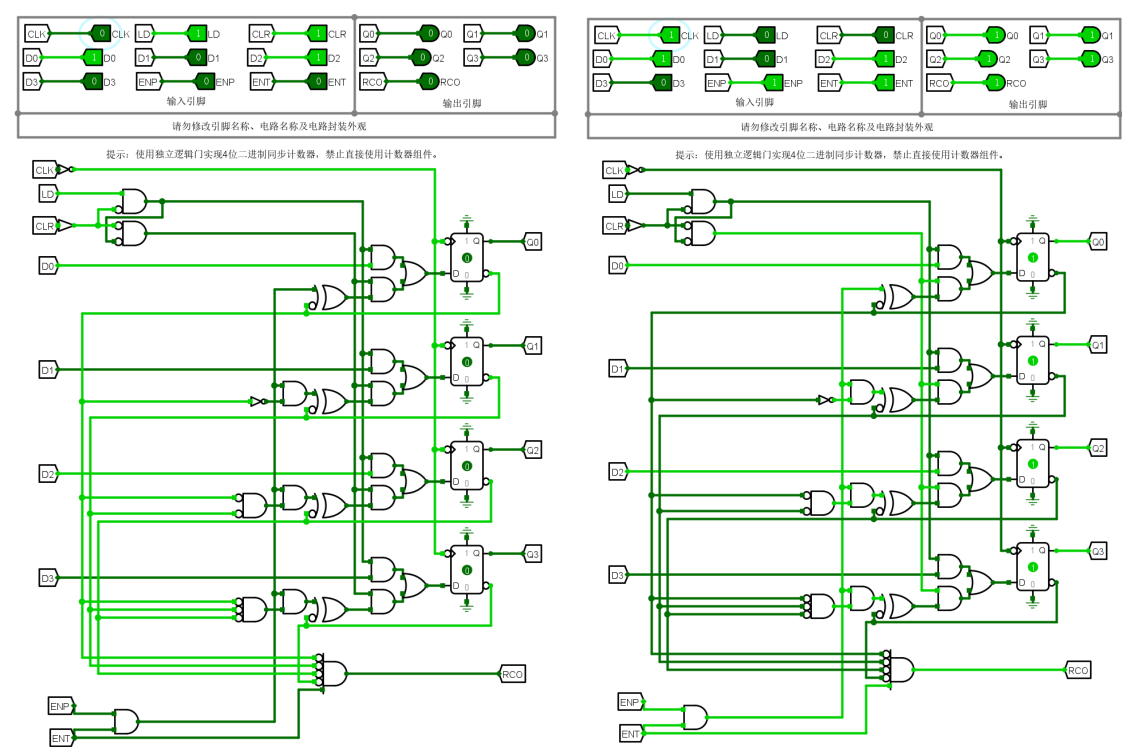


图 3: CLR/END

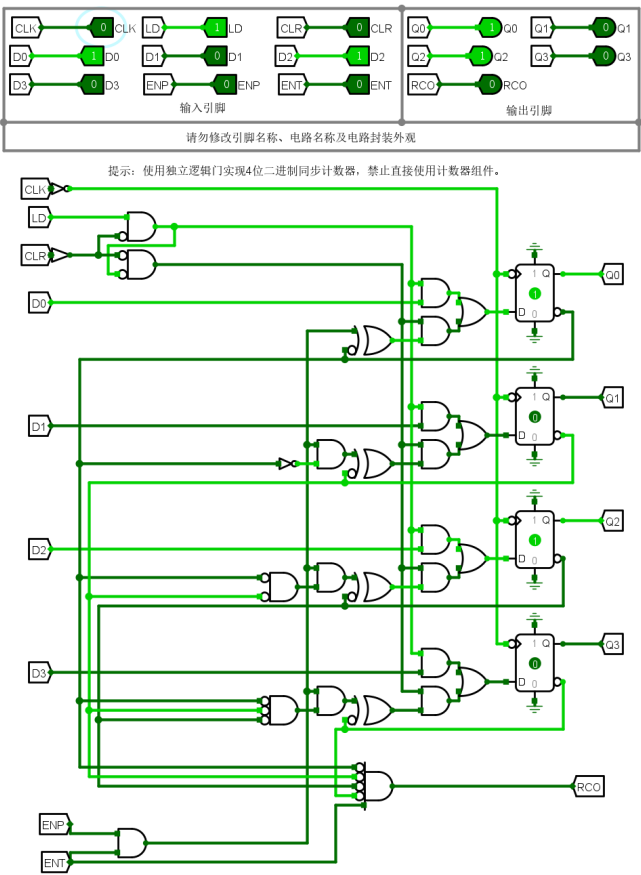


图 4: LD

1.4 错误分析

本次实验没有遇到问题.

2 4 位通用移位寄存器

2.1 整体方案设计

本实验使用 4 个 D 触发器和若干组选择逻辑构成 4 位通用移位寄存器。寄存器支持并行装载、左移、右移和保持四种工作方式，同时提供左右两端串行输入 LIN 与 RIN ，并带有低电平有效的异步清零端 CLR_L 。

端口定义：

端口符号	端口功能
输入端 CLK	时钟输入端
输入端 CLR_L	异步清零端，低电平有效
输入端 S_1, S_0	工作方式选择端
输入端 $D_3 \sim D_0$	并行输入端
输入端 LIN, RIN	左移、右移串行输入端
输出端 $Q_3 \sim Q_0$	4 位输出端

其工作方式可概括为： $S_1S_0 = 00$ 时保持，01 时右移，10 时左移，11 时并行装载。这样既能完成数据的串并转换，也能作为后续时序部件的通用寄存单元。

2.2 实验电路图

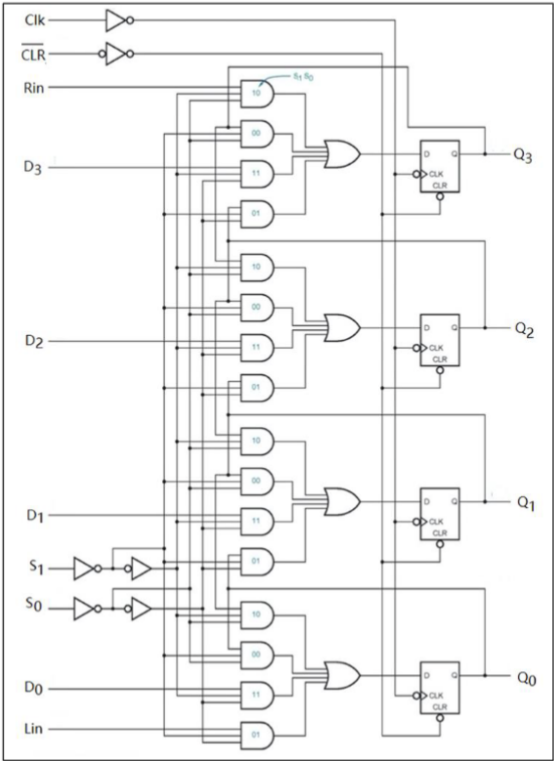
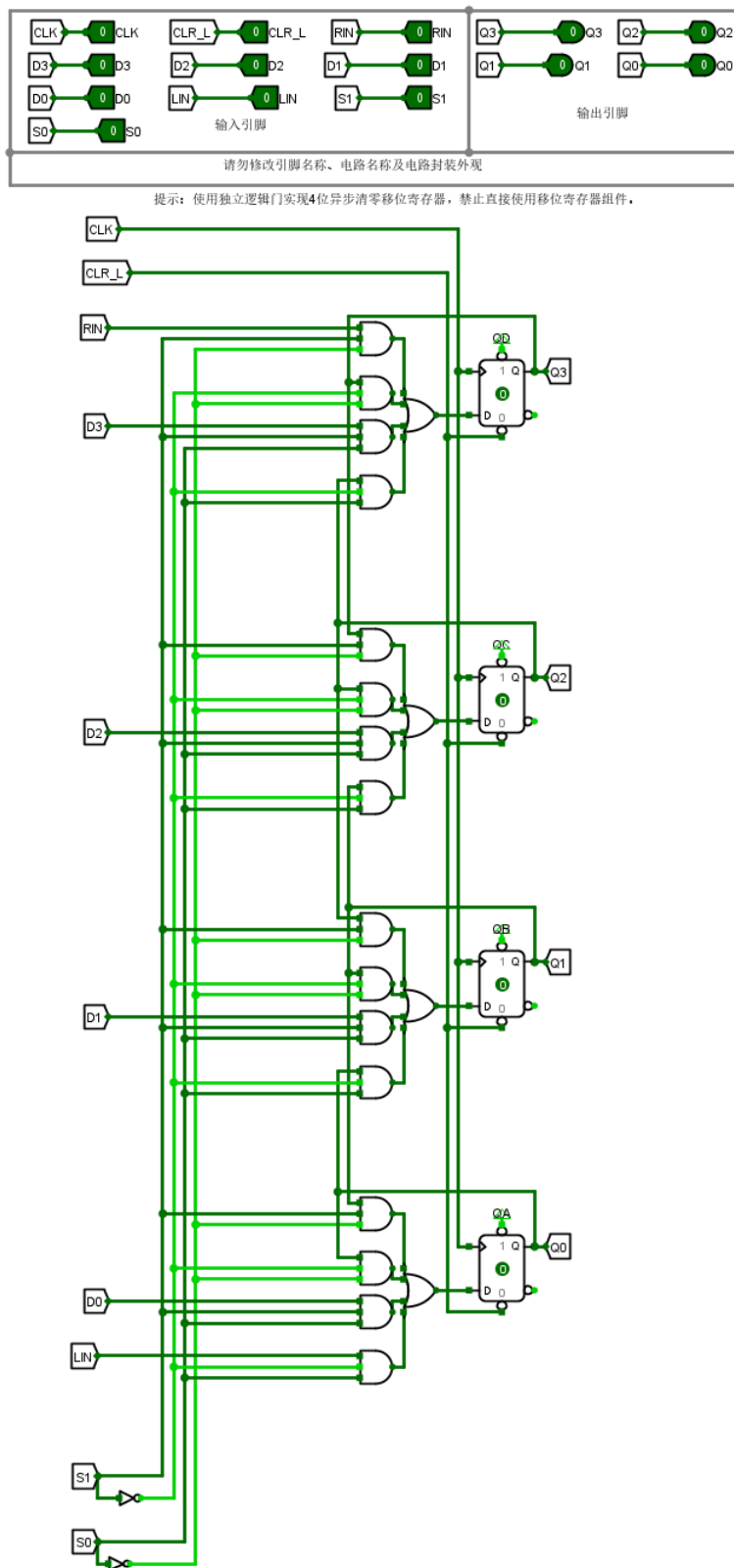


图 5: 4 位通用移位寄存器原理图



2.3 仿真测试

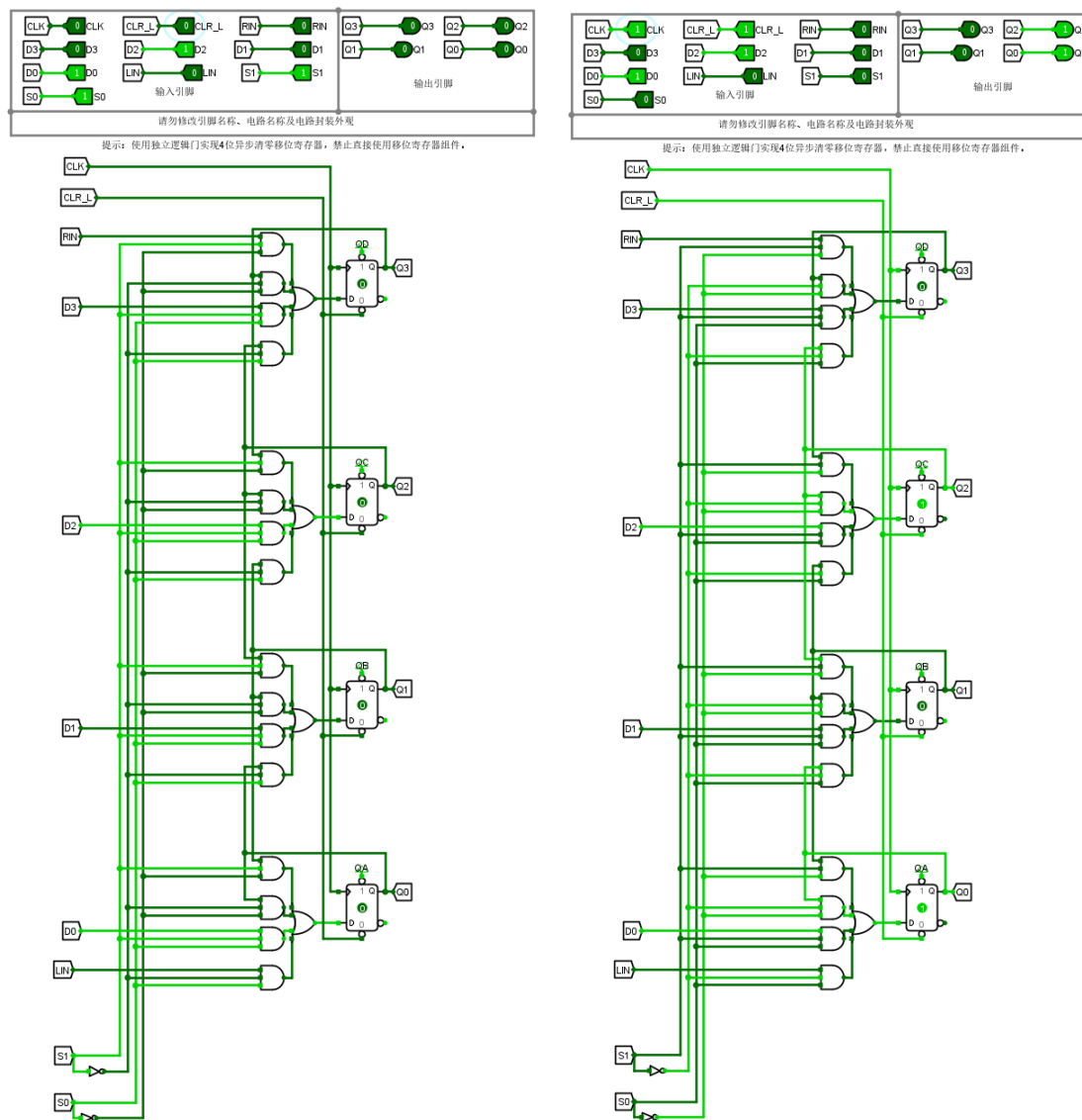


图 7: CLR/KEEP

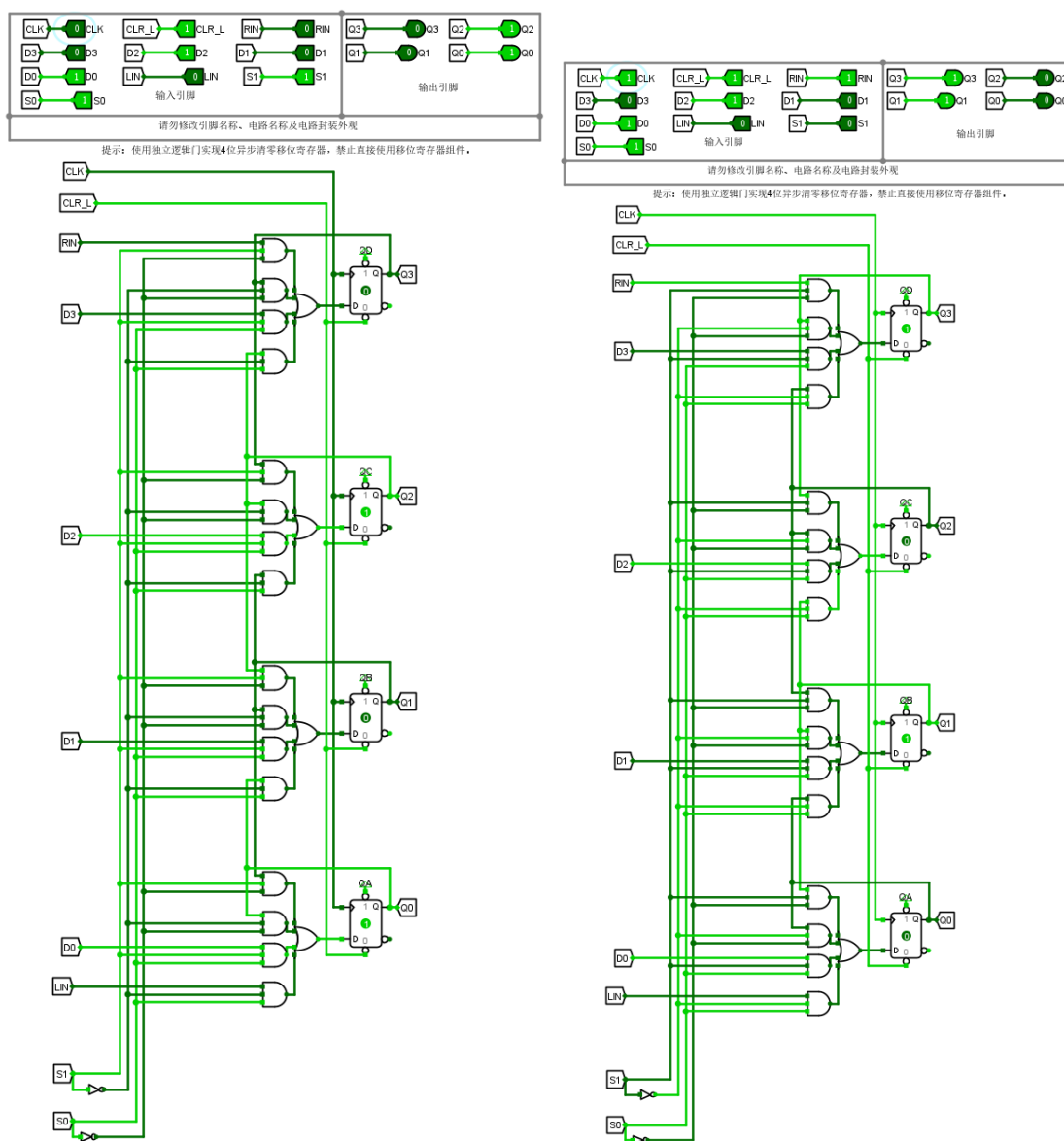


图 8: LD/LIN

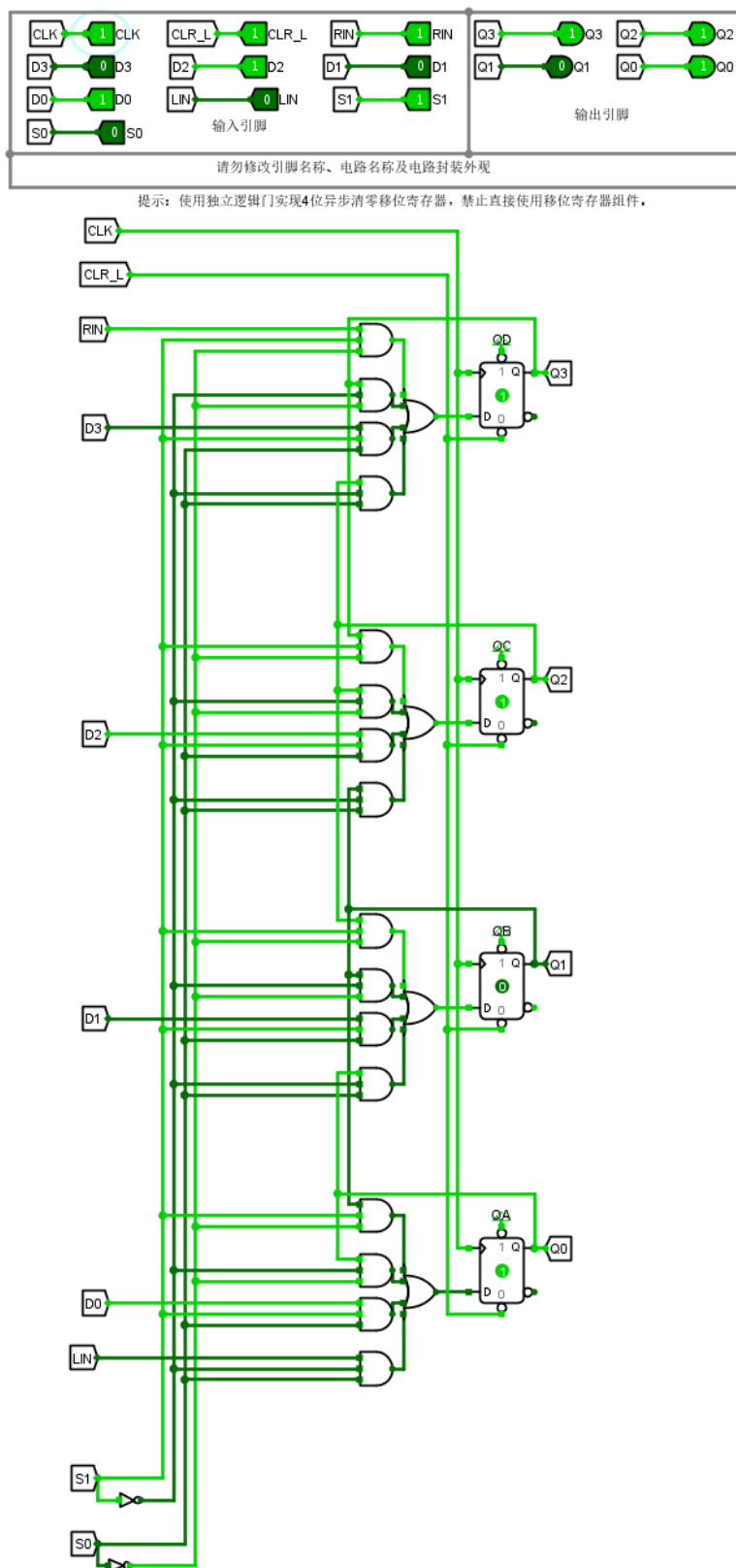


图 9: RIN

2.4 错误分析

本次实验没有遇到问题.

3 4 位无符号乘法器

3.1 整体方案设计

本实验采用移位加法的方式实现 4 位无符号乘法器。电路以被乘数 X 、乘数 Y 和结果寄存器 Z 为核心，配合 4 位同步计数器控制 4 个计算周期：每个周期根据乘数最低位决定是否将当前部分积与被乘数相加，再通过移位单元更新部分积，直到完成全部位的运算。

端口定义：

端口符号	端口功能
输入端 CLK	时钟输入端
输入端 RST	复位端
输入端 X	被乘数输入端
输入端 Y	乘数输入端
输出端 Z	8 位乘积输出端

3.2 实验电路图

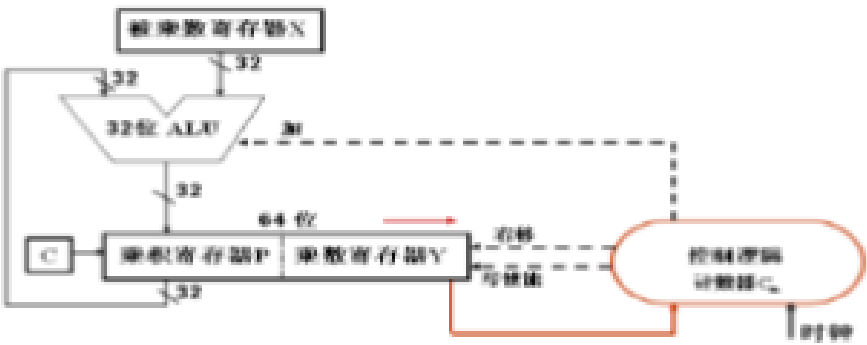


图 10: 4 位无符号乘法器原理图

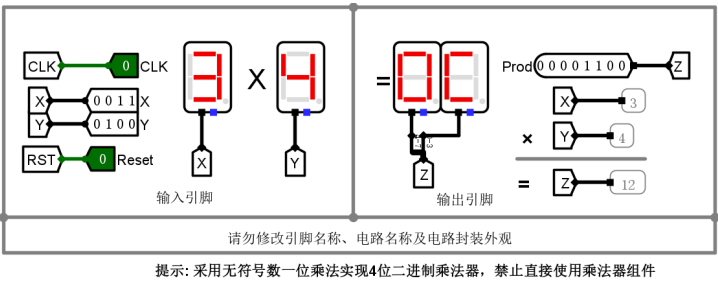


图 11: 4 位无符号乘法器电路图

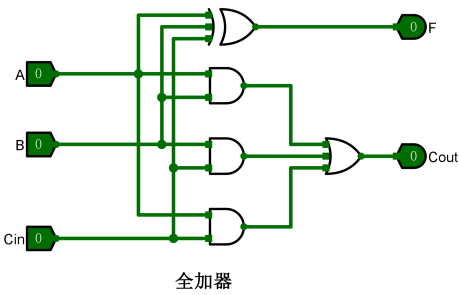


图 12: 全加器电路图

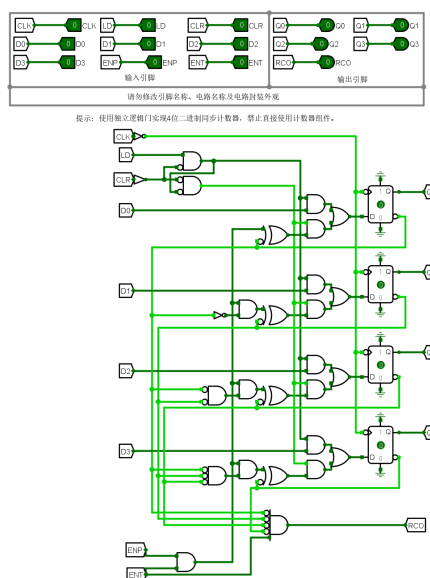


图 13: 4 位同步计数器电路图

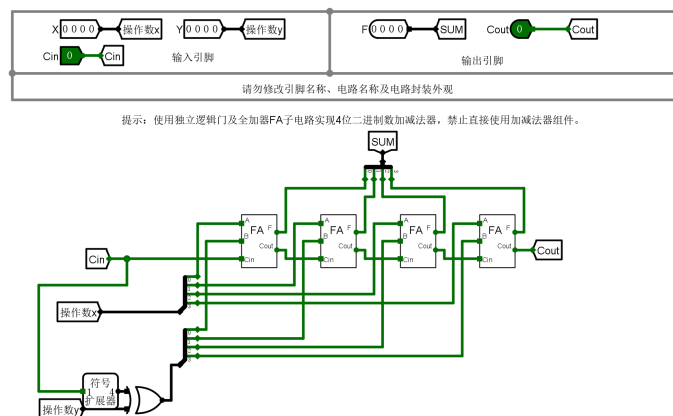
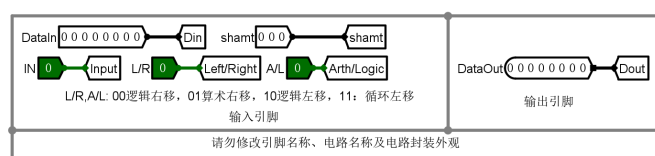


图 14: 4 位加法器电路图



提示：使用多路选择器级联实现8位桶形移位器，禁止直接使用移位器组件。

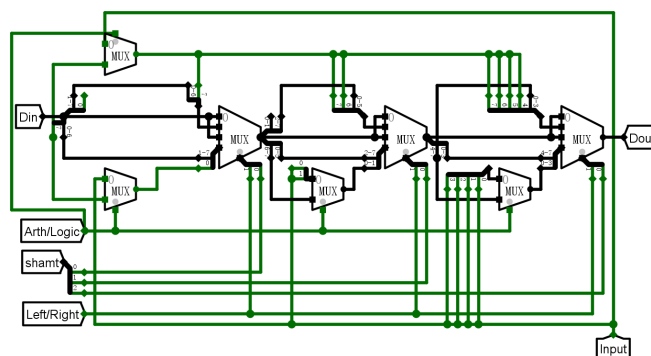


图 15: 8 位移位器电路图

3.3 仿真测试

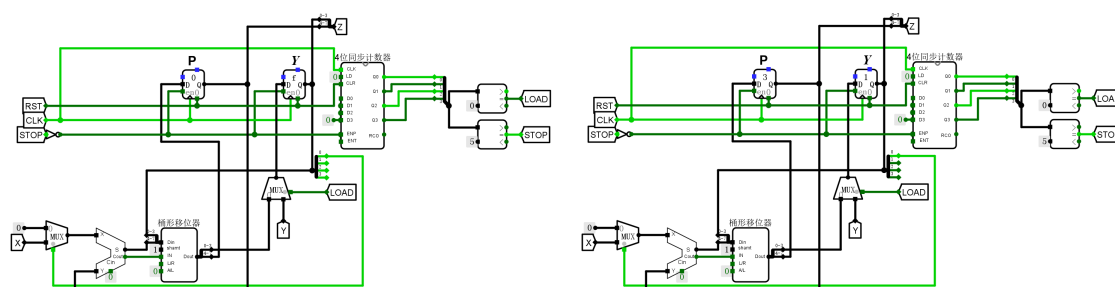
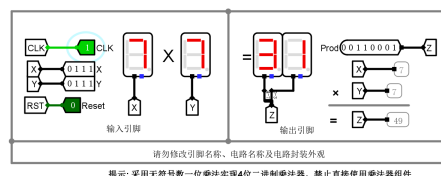
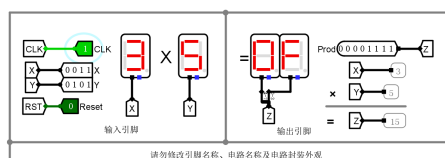


图 16: 4 位无符号乘法器仿真测试图（测试 1/测试 2）

3.4 错误分析

一开始容易将计数器的两个使能端直接接 1，导致乘法器不能正确停止；然而，测试平台上这样连接的电路依然能够通过测试。

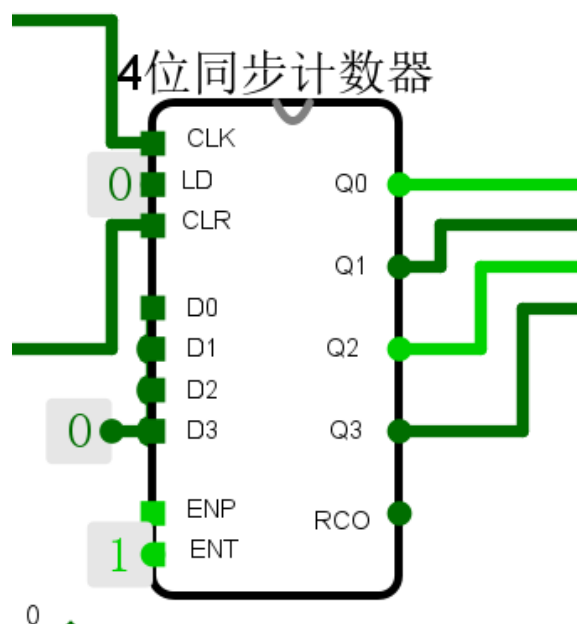


图 17: 错误连接的计数器

实际上，这里移位器的引入显得多余，因为每次都只移动固定的位数，只需要分线器就可以完成移位。

4 32 位寄存器堆

4.1 整体方案设计

端口定义：

端口符号	端口功能
输入端 CLK	时钟输入端
输入端 WE	写使能端
输入端 RW	写地址端
输入端 RA, RB	两个读地址端
输入端 Din	写入数据端
输出端 RDA, RDB	两个读数据端

4.2 实验电路图

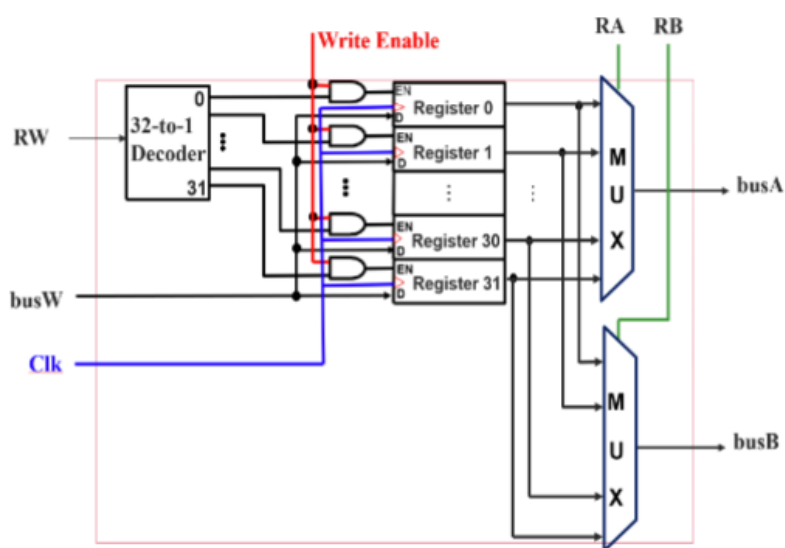


图 18: 32 位寄存器堆原理图

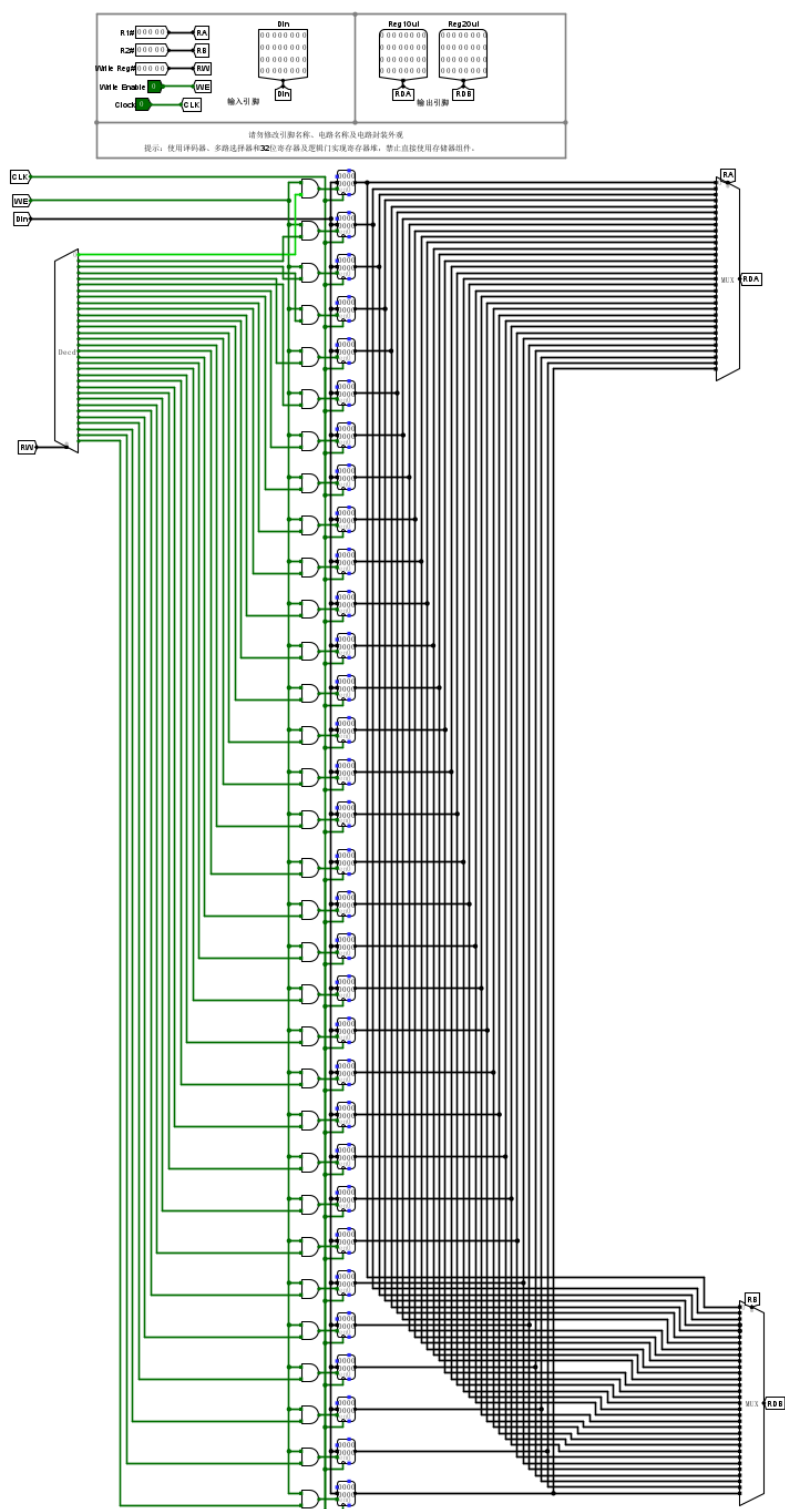


图 19: 32 位寄存器堆内部电路图

4.3 仿真测试

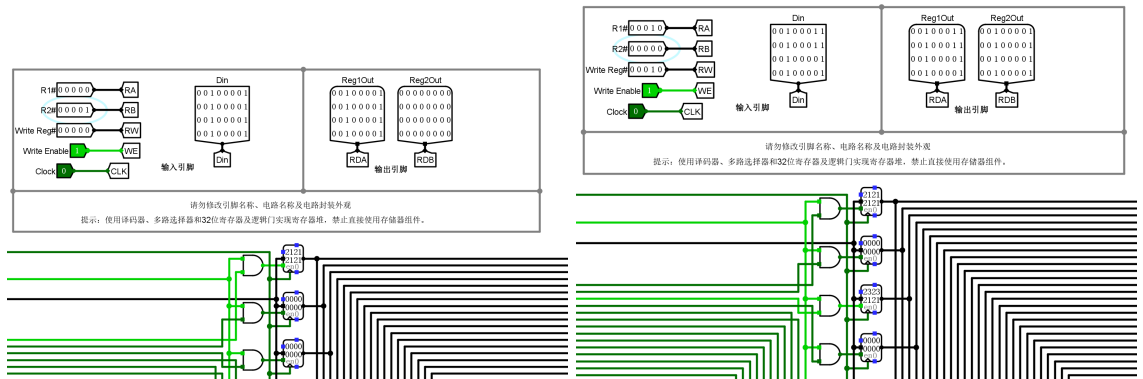


图 20: 32 位寄存器堆仿真测试图（测试 1/测试 2）

4.4 错误分析

本次实验没有遇到问题.

5 数字时钟

5.1 整体方案设计

端口定义:

端口符号	端口功能
输入端 <i>CLK</i>	时钟输入端
输入端 <i>LD</i>	初始值装载端
输入端 <i>START, STOP</i>	启停控制端
输出端 <i>H1, H0, M1, M0, S1, S0</i>	时、分、秒的 BCD 输出端
输入端 <i>HH1, HH0, MM1, MM0, SS1, SS0</i>	时、分、秒的 BCD 输入端
输出端 <i>LED_R, LED_G, LED_B</i>	RGB 指示灯输出端
输出端 <i>INErr</i>	非法输入检测输出端

5.2 实验电路图

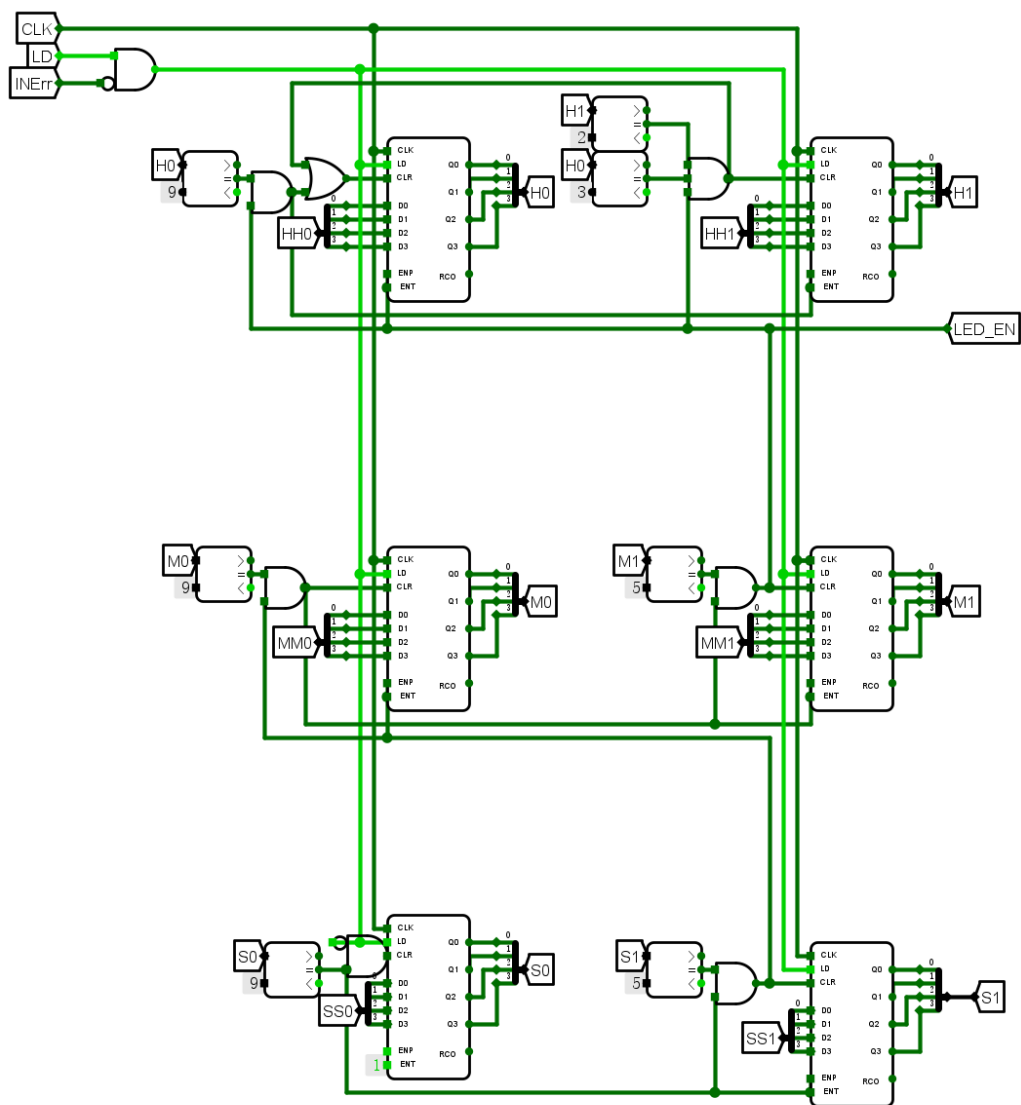


图 21: 数字时钟计时电路

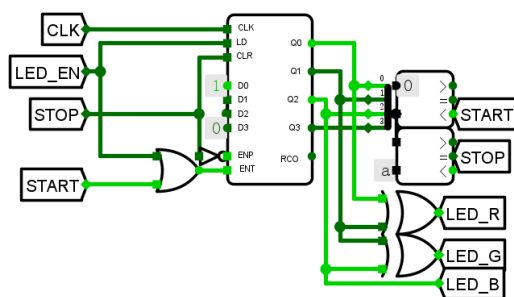
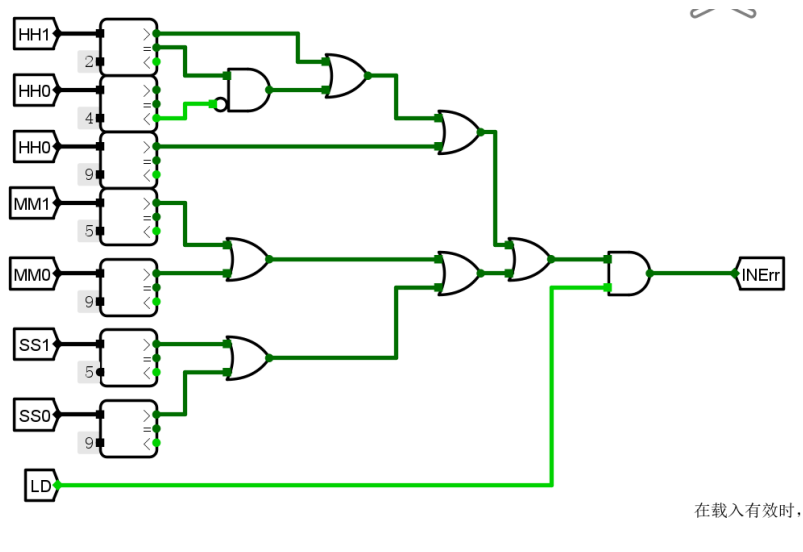


图 22: 数字时钟非法输入检测和 RGB 指示电路

5.3 仿真测试

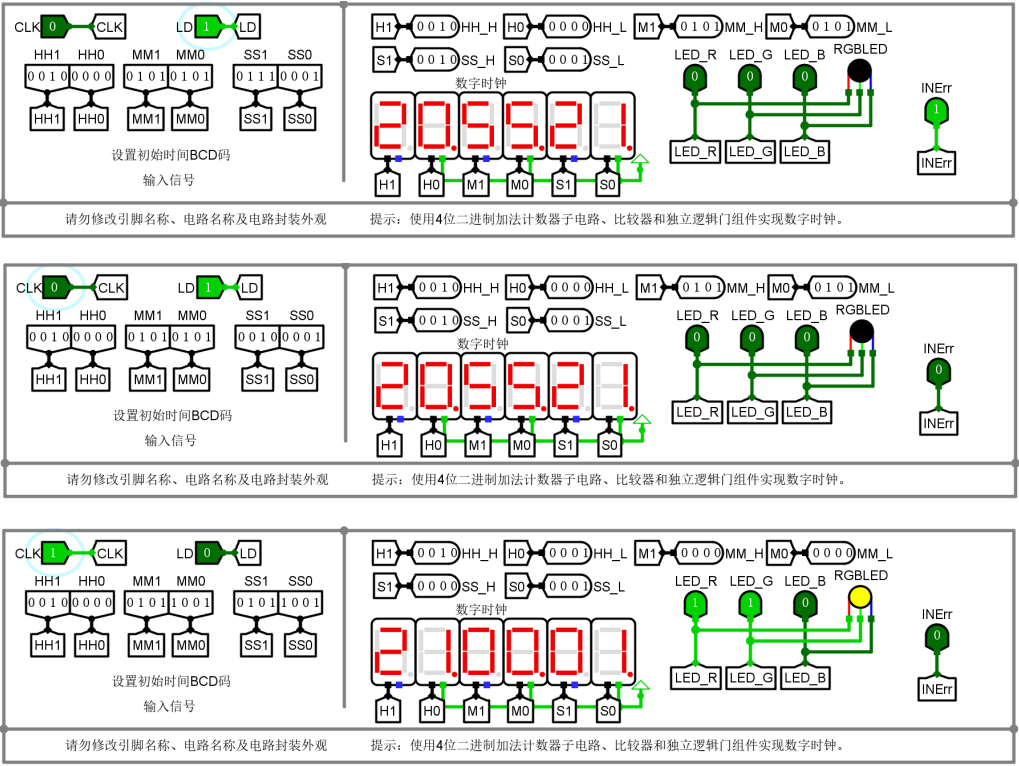


图 23: 数字时钟仿真测试图 (INErr/LD/RGB)

5.4 错误分析

一开始因为检测非法输入的电路框里没有 LD 隧道，我误以为只需要检测输入的时分秒是否符合范围即可，导致有些测试无法通过；将 LD 信号加入 INErr 判断后，测试才得以通过。

关于 RGB 指示灯的颜色变化令人感到困惑，没有明确的提示（例如品红/青色是由哪两种或三种颜色混合而成的），最终只能通过反复测试来总结规律，浪费了不少时间。

6 思考题

6.1

使用触发器的置位端和复位端。

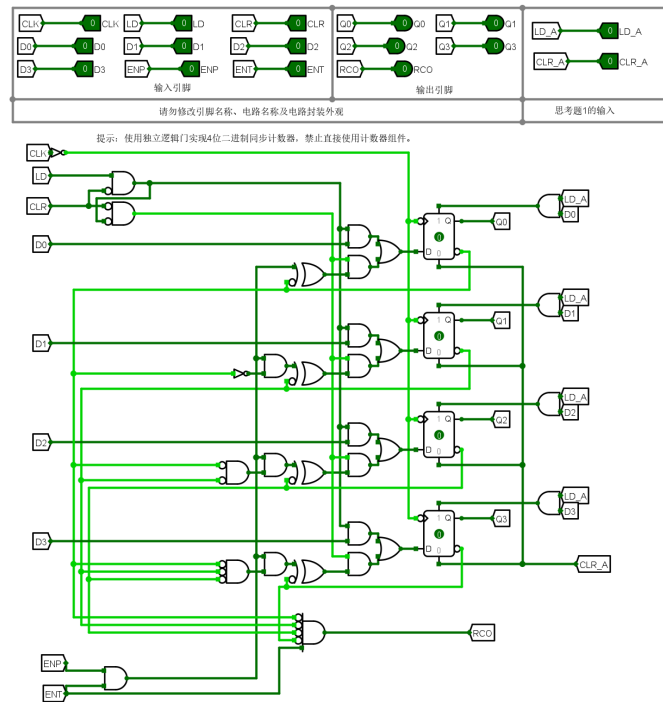


图 24: 异步置位/复位的计数器

6.2

修改 RCO 前与门的端口，使其在 10101 时输出 1，再将 RCO 连接到触发器的复位端，可实现模 10 计数器。

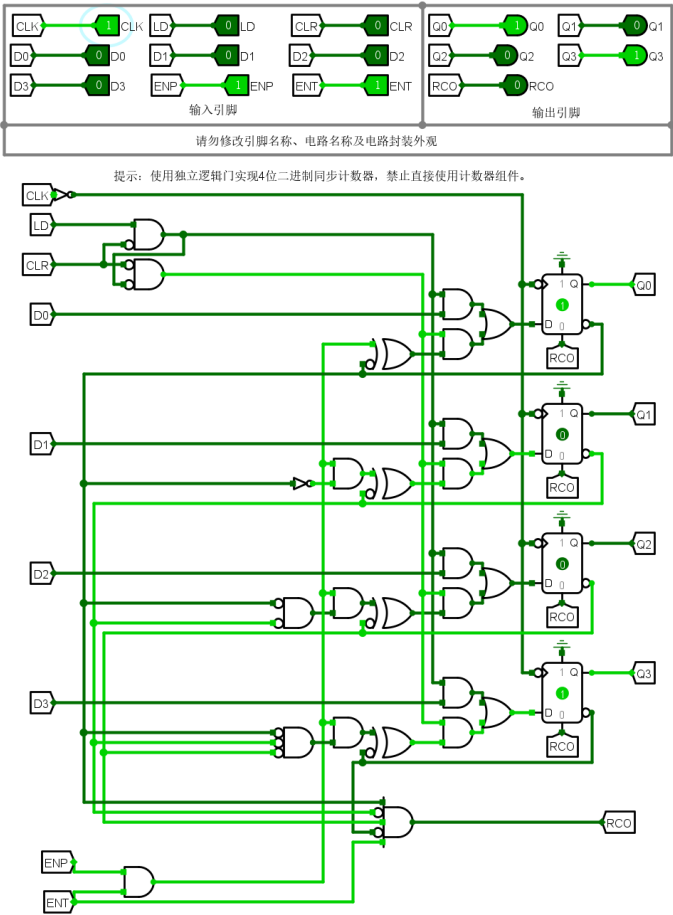


图 25: 模 10 计数器

6.3

使用 LFSR 算法，将两个 4 位寄存器连接成一个 8 位寄存器，并设计相应的组合逻辑实现伪随机数的生成。

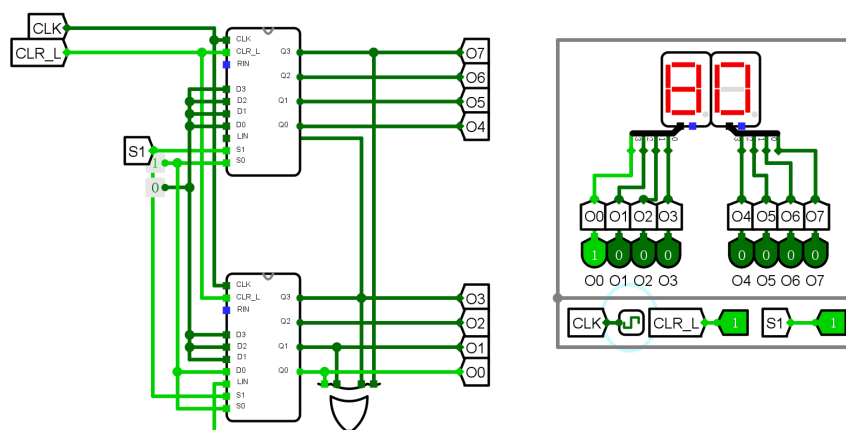


图 26: 伪随机数生成器

复位: $CLR_L = 1, S1 = 1$ 时, 置位 01H。工作: $CLR_L = 1, S1 = 0$ 时, 随 CLK 更新状态。

输出序列 (十六进制):

$80 \rightarrow c0 \rightarrow 60 \rightarrow b0 \rightarrow 58 \rightarrow 2c \rightarrow 16 \rightarrow 8b \rightarrow 45 \rightarrow 22 \rightarrow 11 \rightarrow 08 \rightarrow 04 \rightarrow 02 \rightarrow 01$

可以改变 4 输入异或门连接的端口改变序列长度。

6.4

说实话我不是很理解这里“写后读”的含义。

当前设计中的寄存器堆会在写使能 WE 为高电平且时钟沿上升时写入 D_{in} 数据, 同时 RDA 和 RDB 直接反映写入后寄存器的值。

如果希望 RDA 和 RDB 不直接反映写入的数据, 可以在 RDA 和多路选择器间加入一个寄存器, 只有在写入数据的下一个时钟沿才更新这个寄存器的值, 这样可以实现“写后读”的效果。

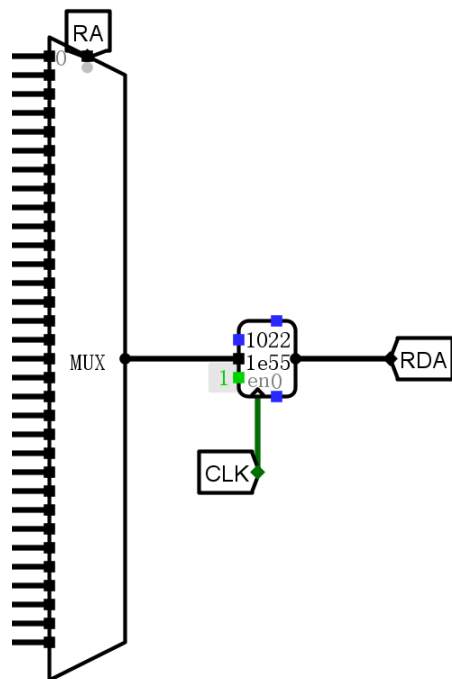


图 27: 修改后的寄存器堆部分电路图